

BASIC SKELETON OF C++ PROGRAM

Let's vividly look at the following C++ Program and understand every command used in order to give us a clear picture of how our programs look like throughout this lecture.

```
1. #include <iostream>
2. using namespace std;
3.
4. int main()
5. { cout << "Hello World!";
6.   return 0;
7. }
```

Code explained

Line 1: #include <iostream> is a **header file library** that lets us work with input and output objects, such as **cout** (used in line 5). Header files add functionality to C++ programs.

Line 2: using namespace std means that we can use names for objects and variables from the standard library.

Don't worry if you don't understand how #include <iostream> and using namespace std works. Just think of it as something that (almost) always appears in your program.

Line 3: A blank line. C++ ignores white space.

Line 4: Another thing that always appear in a C++ program, is **int main()** . This is called a **function**. Any code inside its curly brackets { } will be executed.

Line 5: cout (pronounced "see-out") is an **object** used together with the **insertion operator (<<)** to output/print text. In our example it will output "Hello World".

Note: Every C++ statement ends with a semicolon;

Note: The body of **int main()** could also been written as:

```
int main () { cout << "Hello World! "; return 0; }
```

Remember: The compiler ignores white spaces. However, multiple lines make the code more readable.

Line 6: `return 0` ends the main function.

Line 7: Do not forget to add the closing curly bracket `}` to actually end the main function.

The C++ **Cout** command that display your output on the screen

Example 1.0:

```
#include <iostream>
using namespace std;
```

```
int main() {
    cout << "This is My First Code in C++";
    return 0;
}
```

The **cout** object, together with the Insertion `<<` operator, is used to output values/print text while the **cout** object with the extraction operator (`>>`) is used to receive an input from the keyboard:

C++ Output (Print Text)

The **cout** object, together with the `<<` operator, is used to output values/print text:

Example:

```
#include <iostream>
using namespace std;
int main() {
    cout << "COM 311 CLASS!";
    return 0;
}
```

You can add as many cout objects as you want. However, note that it does not insert a new line at the end of the output:

Example:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Federal Polytechnic Mubi";
    cout << "Yola Campus";
    return 0;}
```

New Lines

To insert a new line, you can use the \n character:

Example

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World! \n";
    cout << "I am learning C++";
    return 0;
}
```

OR

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World!"<<endl;
    cout << "I am learning C++";
    return 0;
}
```

Tip: Two \n characters after each other will create a blank line:

Example:
#include <iostream>
using namespace std;

```
int main() {
    cout << "Hello World! \n\n";
    cout << "I am learning C++";
    return 0;
}
```

Both `\n` and `endl` are used to break lines. However, `\n` is most used.

But what is `\n` exactly?

The newline character (`\n`) is called an **escape sequence**, and it forces the cursor to change its position to the beginning of the next line on the screen. This results in a new line.

Examples of other valid escape sequences are:

Escape Sequence	Description
<code>\t</code>	Create Horizontal tab
<code>\\</code>	Create a backslash Character (<code>\</code>)
<code>\"</code>	Insert a double quote character

Exercise:

Write a C++ program using the escape Sequences `\t` and `\n` to display the following output:

```
S/No      Description      Remark
1.        Maggi Cube      Cost: 1100
2.        Sugar Cube      Cost: 1100
3.        Peak Powder      Cost: 1100

Process returned 0 (0x0)   execution time : 0.034 s
Press any key to continue . . .
```

C++ Comments

Comments can be used to explain C++ code, and to make it more readable. It can also be used to prevent execution when testing alternative code. Comments can be single-lined or multi-lined..

Single-line Comments

Single-line comments start with two forward slashes (//).

Any text between // and the end of the line is ignored by the compiler (will not be executed).

This example uses a single-line comment before a line of code:

```
// This is a comment  
cout << "Hello Class";
```

Or

```
cout << "Hello World!"; // This is a comment
```

C++ Multi-line Comments

Multi-line comments start with /* and ends with */.

Any text between /* and */ will be ignored by the compiler:

Example

```
/* The following line of code would print  
Hello Class to the screen*/
```

```
cout << "Hello World!";
```

C++ Variable Types

A variable is a name which is associated with a value that can be changed. For example when I write `int value{38}`; here variable name is value which is associated with value 33, `int` is a data type that represents that this variable can hold integer values. The name of a variable can be composed of letters, digits, and the underscore character. It must begin with either a letter or an underscore. Upper and lowercase letters are distinct because C++ is case-sensitive

Rules for Assigning Name to a Variable in C++

1. All variable names must begin with a letter of the alphabet or an underscore (`_`).
2. After the first initial letter, variable names can also contain letters and numbers.
3. Uppercase characters are distinct from lowercase characters; this is because C++ is a case sensitive language. A variable with name `hours` is different with `HOURS` in C++
4. You cannot use a C++ keyword (reserved word) as a variable name. (e.g `int`, `bool`, `char` cannot be used as variable name in C++ but if you want to use one such as the `int` then you have to add a prefix such as `int_score`).
5. Variable names in C++ can range from 1 to 255 characters
6. No spaces or special characters are allowed.